INVENTORY MANAGEMENT

MR.PIBHU CHITBURANACHART
MR.PANNAWISH KRIENGYAKUL
MR.AKKHRAWIN NAIR

A PROJECT REPORT SUBMITTED IN PARTIAL FULFILLMENT OF THE REQUIREMENT FOR THE DEGREE BACHELOR OF ENGINEERING IN COMPUTER ENGINEERING

FACULTY OF ENGINEERING & INTERNATIONAL COLLEGE
MAHIDOL UNIVERSITY

2026

INVENTORY MANAGEMENT

MR.PIBHU CHITBURANACHART
MR.PANNAWISH KRIENGYAKUL
MR.AKKHRAWIN NAIR

A PROJECT REPORT SUBMITTED IN PARTIAL
FULFILLMENT OF THE REQUIREMENT FOR THE DEGREE
BACHELOR OF ENGINEERING IN COMPUTER ENGINEERING

FACULTY OF ENGINEERING & INTERNATIONAL COLLEGE
MAHIDOL UNIVERSITY

2026

Computer Engineering Project
entitled

INVENTORY MANAGEMENT

Mr.Pibhu Chitburanachart
Researcher

Mr.Pannawish Kriengyakul
Researcher

Mr.Akkhrawin Nair
Researcher

Asst. Prof. Mingmanas Sivaraksa, Ph.D. (Data Science),
Ph.D. (Computer Engineering)
Advisor

Thesis
entitled
INVENTORY MANAGEMENT

was submitted to the Faculty of Engineering & International College,
Mahidiol University
for the degree of Bachelor of Engineering (Computer Engineering)
on
July 15, 2024

_____
Asst. Prof. Mingmanas Sivaraksa, Ph.D. (Data Science),
Ph.D. (Computer Engineering)
Chair Committee

_____
Committee 1, Ph.D. (Computer Engineering)
Committee

_____
Committee 2, Ph.D. (Computer Engineering)
Comittee

BIOGRAPHY

NAME	Mr.Pibhu Chitburanachart
DATE OF BIRTH	29 August 2004
BIRTHPLACE	Bangkok, Thailand
EDUCATION BACKGROUND	Mahidol International College
MOBILE PHONE	098 494 7984
E-MAIL	Pibhu22@gmail.com

NAME	Mr.Pannawish Kriengyakul
DATE OF BIRTH	17 April 2003
BIRTHPLACE	Bangkok, Thailand
EDUCATION BACKGROUND	Mahidol International College
MOBILE PHONE	0989705604
E-MAIL	pannawish.kri@gmail.com

BIOGRAPHY

NAME	Mr.Akkhrawin Nair
DATE OF BIRTH	22 September 2002
BIRTHPLACE	Bangkok, Thailand
EDUCATION BACKGROUND	Mahidol International College
MOBILE PHONE	0987819644
E-MAIL	akkhrawin22@gmail.com

ACKNOWLEDGEMENT

First and foremost, we would like to express our special thanks and sincere of gratitude to our advisor and co-advisors, Asst. Prof. Mingmanas Sivaraksa, Asst. Prof. Lalita Narupiyakul, Asst. Prof. Tanasanee Phienthrakul, who insight, advise and knowledge us a lot in finalizing this project within the limited time frame.

Beside our advisors, we would like to thank to the committees, Asst. Prof. XXX XXXX, Asst. Prof. YYY YYYY, and Asst. Prof. ZZZ ZZZZ, for their insightful comment and encouragement. Also, the questions which encourage us to rethink and research more about some factors which we missed. Therefore, this project would not be completed without comments, questions, and support from our committees.

Finally, we would like to special thanks to our university, Mahidol University International College, and Faculty of Engineering, Mahidol University, which provided us a lot of resources to study, financial means for researching this project.

Mr.Pibhu Chitburanachart
Mr.Pannawish Kriengyakul
Mr.Akkhrawin Nair

ชื่อโครงงาน	ชื่อวิทยานิพนธ์
ผู้วิจัย	นายพิภู จิตบูรณะชาติ ICCI
	นาย ปัณณวิชญ์ เกรียงยะกุล ICCI
	นายอัครวินทร์ แนร์ ICCI
	ปริญญา วิศวกรรมศาสตรบัณฑิต สาขาวิชาวิศวกรรมคอมพิวเตอร์
อาจารย์ที่ปรึกษา	ผศ.□□. มิ่งมานัส ศิวรักษ์
สำเร็จการศึกษา	15 กรกฎาคม 2567

บทคัดย่อ

บทคัดย่อภาษาไทย (not required for your project proposal, but it is mandatory for the black book)

คำสำคัญ : ลาเท็ก / วิทยานิพนธ์ (~5 คำ)

60 หน้า

INVENTORY MANAGEMENT

STUDENTS	Mr. Pibhu Chitburanachart ICCI
	Mr. Pannawish Kriengyakul ICCI
	Mr. Akkhrawin Nair ICCI
DEGREE	Bachelor of Engineering (Computer Engineering)
PROJECT ADVISOR	Asst. Prof. Mingmanas Sivaraksa, Ph.D. (Data Science), Ph.D. (Computer Engineering)
DATE OF GRADUATION	July 15, 2024

ABSTRACT

This project presents an Inventory Management System for SME trading businesses that buy from suppliers and resell to customers. The system brings products, categories, suppliers, customers, purchases, sales, quotations, billing notes, payment batches, credit notes, inventory checking, dashboard summaries, and a read-only AI assistant into one web application.

The application was developed with React and Vite on the frontend, Django REST Framework on the backend, and MySQL as the database. The backend controls important rules such as stock calculation, sale stock validation, and eligibility for billing notes, payment batches, and credit notes.

The completed system improves stock visibility and connects purchasing, sales, and financial document tracking. The AI assistant is limited to supported read-only questions about stock, partners, receivables, payables, and document references.

60 Pages

CONTENTS

LIST OF TABLES

LIST OF FIGURES

CHAPTER 1
INTRODUCTION

1.1 Background

Small and medium-sized trading businesses often operate as intermediaries between suppliers and customers [5]. In daily work, the business has to buy products from suppliers, receive stock, sell products to customers, prepare documents, and check whether the sales price still gives a reasonable margin. Because of this, inventory management is not only about counting how many products are left in the warehouse. It also involves quotations, purchase orders, sales records, delivery status, receivables, payables, and stock availability.

Many small businesses still depend on spreadsheets, paper records, manual document folders, or general-purpose accounting tools to manage these tasks [1, 6]. This can work at the beginning, but it becomes harder when the business has more products, more suppliers, different product units, delayed purchases, cancelled items, customer billing, and supplier payments. It also becomes difficult to answer simple questions quickly, such as which products are low in stock, which sales can be billed, which purchases are ready for payment, or whether there is enough received stock to fulfill a sale.

This project was created to bring those daily tasks into one web application. The Inventory Management System focuses on the main workflow of a trading business: managing master data, recording purchases and sales, checking stock, preparing quotations, tracking billing notes and payment batches, handling credit notes, viewing dashboard summaries, and asking limited read-only questions through the AI assistant.

1.2 Objective

1. To develop a system that records purchase and sales transactions, including the current status of each process, with an attachment file function that links all supporting documents directly to the corresponding transactions.

2. To develop an inventory management system that displays the overall product stock in the warehouse and provides tools for calculating essential inventory information.
3. To integrate an AI chatbot using the ChatGPT API that connects to the system's backend data, allowing users to request supported operational summaries through text queries.

1.3 Scope

The scope of this project is based on the features that are already implemented in the Inventory Management System codebase. The system covers the following areas:

1. **Master data management:** The system manages categories, products, suppliers, and customers. Product records include SKUs, previous SKUs, product names, categories, unit conversions, reorder levels, active or disabled status, product pictures, and supplier-specific sourcing information.
2. **Purchasing workflow:** The system records purchase transactions, supplier information, purchase line items, expected delivery dates, received dates, item statuses, VAT totals, bill discounts, supplier tax invoice references, uploaded documents, and payable totals. The backend keeps purchase status and purchase item status in sync.
3. **Sales workflow:** The system records sales transactions, customer information, customer purchase-order references, line items, delivery statuses, uploaded documents, VAT totals, discounts, billing-note links, credit-note links, and source quotation links. The backend checks stock before a sale can move into a stock-deducting status.
4. **Inventory and stock control:** Available stock is calculated from received purchase quantities minus committed sale quantities. Sale allocations use received purchase layers and support first-in, first-out allocation. The inventory page and dashboard show stock position, reorder information, demand, purchase pipeline, stock value, and product history.
5. **Quotation workflow:** The system stores quotations with customer and supplier references, validity information, shipping date, payment term, VAT mode, to-

tal amounts, line items, and supplier options. Quotations can also be linked to purchases and sales created from them.

6. **Finance workflow:** The system supports billing notes for customer receivables, payment batches for supplier payables, and credit notes for cancelled or returned sales lines. Backend eligibility endpoints prevent users from selecting invalid transactions for these finance documents.
7. **Dashboard and reporting:** The backend provides dashboard summary and segment endpoints for finance, trends, product movement, cashflow, and order coverage.
8. **Authentication and user access flow:** The backend includes JWT login, refresh, and authenticated user profile endpoints. The frontend supports login, token refresh, logout, and guest mode. The system also includes a user-access page for managing users, roles, and role permissions. Backend-wide authentication enforcement is controlled by the INVENTORY_REQUIRE_AUTH setting, and protected API actions use Django model permissions when authentication enforcement is enabled.
9. **AI-assisted operation:** The system includes a text-based inventory assistant endpoint for read-only questions covering stock and fulfillment, customer or supplier summaries, receivables and payables with exception checks, and reference or line-item lookup. It also includes an AI report page that generates printable business reports for a selected supplier, customer, or product.

1.4 Expected Results

The expected result of this project is a working web-based inventory management system that can record purchases and sales, track the status of each process, and attach supporting documents to the correct transaction. The system should also show stock levels based on received purchases and committed sales, so users can understand where the stock comes from and where it is used.

The system is also expected to make daily checking easier through dashboard summaries and the limited AI-assisted text interface. Users should be able to find supported information such as product stock, transaction references, receivables, payables,

partner summaries, and stock exceptions without manually searching every page. Overall, the completed system should reduce manual checking, improve stock visibility, and support better daily decisions for an SME trading business.

1.5 Timeline

Table 1.1 Project Timeline

Plan	Timeline									
	2025				2026					
	Sep	Oct	Nov	Dec	Jan	Feb	Mar	Apr	May	Jun
1: Initiation	■									
2: Data Analysis		■	■	■	■	■				
3: Requirements			■	■	■	■	■	■		
4: Data Setup				■	■	■	■	■		
5: Development						■	■	■	■	
6: Integration						■	■	■	■	
7: User Test							■	■	■	
8: Final Draft										■
9: Submission										■

CHAPTER 2
LITERATURE REVIEW

This chapter presents a review of references related to the Inventory Management System. The reviewed works cover trading business workflow, spreadsheet-based business records, web-based inventory systems, inventory management concepts, and AI-assisted query support. These topics are important because the project is not only a stock-counting system. It also connects supplier purchasing, customer sales, quotations, receivables, payables, dashboard summaries, and a limited read-only AI assistant.

2.1 North American Industry Classification System: Wholesale Trade [5]

The North American Industry Classification System describes wholesale trade as an intermediate step in the distribution of merchandise. Wholesalers are organized to sell or arrange the purchase and sale of goods for resale, durable non-consumer goods, and raw or intermediate materials. This reference supports the business background of this project because the system is designed for a trading business that works between suppliers and customers.

In this project, the same idea appears in the main workflow. The business buys products from suppliers, stores received stock, and sells products to customers. Therefore, the system needs to manage both sides of the transaction. Supplier data, purchase records, customer data, sales records, and stock availability must be connected so that the user can understand the full flow of goods and money.

2.2 Qtier-Rapor: Managing Spreadsheet Systems and Improving Corporate Performance, Compliance and Governance [1]

This paper discusses how businesses often use spreadsheet systems to fill gaps in their core administrative systems. Spreadsheets are flexible and easy to start with,

but the paper explains that larger spreadsheet systems can become harder to manage and control. This is relevant to small and medium-sized businesses because many businesses begin with simple spreadsheets before moving to a more structured system.

The problem is similar to the motivation of this project. A trading business may be able to record products, purchases, sales, and payments in spreadsheets at first. However, when the number of products, suppliers, customers, and transaction statuses increases, the records become harder to check. This supports the need for a web application with a database, backend validation, and clearer workflow separation.

2.3 Sole Traders Still Not Prepared for Digital Tax Returns in April [6]

This article reports research from Sage about sole traders and their preparation for digital tax reporting. It shows that many small business owners still use older methods such as spreadsheets, bank statements, and pen-and-paper records for financial administration. Although the article focuses on tax reporting, it is useful for this project because it shows that manual and semi-manual record keeping is still common in small business environments.

For an inventory trading business, the same issue can affect daily operations. If stock, purchasing, sales, billing, and payment information are kept in separate files or folders, it becomes difficult to answer simple questions quickly. Examples include checking which products are low in stock, which sales can be billed, and which purchases are ready for payment. This project addresses that issue by keeping related records in one web-based system.

2.4 A Full-Stack Web Solution for Online FMCG Management System Using MERN: Design, Implementation, and Evaluation [3]

This study presents the development of a full-stack web solution for managing fast-moving consumer goods. It discusses a web-based system with inventory management, order processing, stock updates, billing, and user roles. The reference is related to this project because both systems focus on using a web application to reduce manual business work and improve access to operational records.

The main difference is the technology stack and project scope. The reviewed system uses the MERN stack, while this project uses React for the frontend, Django REST Framework for the backend, and MySQL for the relational database. This project also focuses specifically on SME trading workflows, including purchases, sales, quotations, billing notes, payment batches, credit notes, stock validation, dashboard summaries, and limited AI-assisted operation.

2.5 Essentials of Inventory Management [4]

This book explains that inventory management is more than counting available stock. It includes demand, replenishment, physical stock handling, inventory value, and the business decisions connected to stock. These ideas support the design of the inventory and dashboard features in this project.

In the implemented system, stock is calculated from received purchase quantities and committed sale quantities. The dashboard and inventory page also show reorder-related information, pending demand, pending purchases, average cost, stock value, and product movement. These features follow the general inventory management idea that a business should understand both the physical quantity of stock and the financial meaning of that stock.

2.6 Aisha: A Custom AI Library Chatbot Using the ChatGPT API [2]

This article describes the development of a custom chatbot using the ChatGPT API for a university library. The chatbot was designed to answer user questions using prepared system information and a controlled application context. The reference is useful for this project because it shows how a chatbot can be connected to a specific domain instead of being used as a general-purpose conversation tool.

In this Inventory Management System, the AI assistant follows a limited and read-only scope. It can support questions about stock and fulfillment, customer or supplier summaries, receivables and payables, and reference or line-item lookup. It does not create, approve, edit, cancel, or delete records. This limitation is important because inventory and finance records must remain controlled by the main application workflow

and backend validation.

2.7 Summary

The reviewed references support the main direction of this project. The wholesale trade reference explains why a trading business needs to manage the connection between suppliers, products, and customers. The spreadsheet and small-business references show why manual records, spreadsheets, and separated files can become difficult to manage as business operations grow.

The web-based inventory system reference supports the idea of building an integrated application for stock and transaction workflows. The inventory management book supports the calculation and reporting features, especially stock availability, replenishment, and inventory value. The chatbot reference supports the idea of adding a controlled AI-assisted interface, while also showing why the assistant should be limited to a clear domain.

Based on these works, this project focuses on developing a practical web-based inventory management system for SME trading businesses. The system combines React, Django REST Framework, and MySQL to manage business records in a structured way. It also adds dashboard summaries and a read-only AI assistant to help users find supported operational information more quickly.

CHAPTER 3
METHODOLOGY

This chapter explains how the Inventory Management web application was developed. The focus is not only on the tools used, but also on how each page supports the real workflow of an SME trading business. In this project, we built the system as a React web application connected to a Django REST Framework backend and a MySQL database.

3.1 Development Approach

We developed the system step by step, starting from the data that every other workflow depends on. The first part was master data, such as products, categories, suppliers, and customers. After that, we developed the transaction modules: purchases, sales, and quotations. The finance-related modules, including billing notes, payment batches, and credit notes, were added after the main transactions were already working. Finally, we connected the dashboard, inventory workspace, and AI chat page to the backend data.

The project was separated into three main layers:

1. **Frontend layer:** React and Vite were used to build the web pages, forms, filters, tables, modals, and tab navigation.
2. **Backend layer:** Django REST Framework was used to expose APIs, validate data, calculate stock, control eligibility rules, and prepare dashboard and assistant responses.
3. **Database layer:** MySQL was used through the Django ORM to store relational business records.

This structure helped keep the user interface separate from the business rules. The frontend guides the user, but the backend is still responsible for the important checks such as stock validation, transaction eligibility, and payable amount synchronization.

3.2 Requirement Areas and Web-App Pages

The web application is organized around the main work areas that a trading business needs each day. Each sidebar page has a specific purpose, and the pages are connected through shared product, supplier, customer, purchase, sale, and finance data.

Table 3.1 Requirement Areas and Implemented Pages

Requirement Area	Web-App Page	Purpose in the System
Operation summary	Dashboard	Shows stock, finance, cashflow, product movement, and order coverage summaries.
Inventory control	Inventory	Shows stock health, reorder information, supplier options, product details, and quick purchase-order preparation.
Assistant support	AI Chat	Answers supported read-only questions about stock, fulfillment, partners, AR/AP exceptions, and document references.
Report generation	AI Report	Generates printable supplier, customer, or product reports with metrics, charts, record tables, and AI-written analysis.
Purchasing	Purchases	Records purchase orders, line items, receiving status, documents, VAT totals, discounts, and payable values.
Supplier payment	Payment Batches	Groups eligible purchases for supplier payment and tracks paid or unpaid lines.
Quotation	Quotation	Prepares quotations with line items, supplier options, validity dates, and conversion to purchase or sale records.
Sales	Sales	Records customer sales, stock allocation, delivery status, documents, VAT totals, and credit-note links.

3.3 Development of Each Page

3.3.1 Dashboard Page

The dashboard page was developed as the first screen for checking the business situation quickly. It receives backend dashboard data and shows several summary areas, including stock health, reorder planning, stock cycling, delivery pipeline, purchase planning, popular products, finance, cashflow, and order coverage. The dashboard is also connected to other pages. For example, a low-stock product can lead the user to the inventory or product page, and an order issue can lead to the purchase or sales page.

Table 3.2 Record and Support Page Scope

Requirement Area	Web-App Page	Purpose in the System
Customer billing	Billing Notes	Groups eligible sales into customer billing notes and tracks received or outstanding lines.
Returns and cancellation adjustment	Credit Notes	Creates credit notes from eligible cancelled or returned sale items.
Product records	Products	Manages product details, SKUs, pictures, units, reorder levels, active status, suppliers, and history.
Category records	Categories	Manages product categories using a parent-child category tree.
Supplier records	Supplier	Manages supplier profile, contact information, procurement contact, branches, shipping addresses, and terms.
Customer records	Customer	Manages customer profile, contact information, branches, shipping addresses, and billing terms.

3.3.2 Inventory Page

The inventory page was developed as the main stock-control workspace. It uses the backend stock report rows instead of calculating stock from full transaction lists in the browser. The page includes an inventory control board, a searchable directory, filters for health status, category, supplier, value, and reorder need, product detail views, and a quick purchase-order drawer. This page is important because it gives the user a practical view of what is available, what is incoming, and what may need to be reordered.

3.3.3 AI Chat Page

The AI chat page was developed as a read-only helper. The user can type a question or use example prompts. The backend prepares a compact context from the current system data and returns a structured answer. If an OpenAI API key is configured, the backend can send the local summary and data context to the configured OpenAI model. If not, it returns the local summary. The current assistant scope is limited to four areas: stock and fulfillment, customer or supplier summaries, receivables/payables with exceptions, and reference or line-item lookup. It does not create, edit, approve, cancel, or delete any record.

3.3.4 AI Report Page

The AI report page was developed for generating printable business reports. The user selects a report scope (supplier, customer, or product), chooses a specific record from the existing master data, and optionally sets a custom date range. When the user clicks generate, the frontend sends the request to the /api/ai-reports/ backend endpoint. The backend collects the related purchases, sales, quotations, billing notes, payment batches, credit notes, and stock data for the selected record. It calculates summary metrics, trend charts, top product or partner charts, and record tables. If an OpenAI API key is configured, the backend sends the collected data context to the AI model and receives an HTML report body with executive summary and business analysis. If no API key is available, the backend builds a local report using the same data. The final report is rendered in a new browser tab as a self-contained HTML document with a print button, styled for both screen and print output. The report supports both English and Thai languages.

3.3.5 Purchases Page

The purchases page was developed for supplier-side transactions. Users can create purchase records, add line items, attach documents, edit existing purchases, update purchase status, and update item receiving status. Purchase items can be pending, received, or cancelled. Only received purchase items become available stock. The page also supports search, filters, pagination, VAT mode, item discounts, bill discount, expected delivery dates, received dates, and payment batch links.

3.3.6 Payment Batches Page

The payment batches page was developed for supplier payable tracking. Instead of letting users manually select any purchase, the page loads eligible purchases from the backend. A purchase is eligible only when it passes the backend rules, such as not already being attached to an active payment batch. Each payment batch stores supplier information, planned and actual payment dates, bank reference, total amount, and paid line status.

3.3.7 Quotation Page

The quotation page was developed for preparing offers before purchase or sales records are finalized. It supports quotation line items, supplier cost options, validity settings, shipping date, payment terms, VAT mode, and total values. The page can also start a purchase or sales flow from a quotation. In the database, quotation items are stored in the QuotationItem table, and supplier options are stored in QuotationItemSupplier; however, the API still returns an items array so the frontend can work with a simple structure.

3.3.8 Sales Page

The sales page was developed for customer-side transactions. It records sale headers, customer information, customer PO reference, sale items, documents, VAT totals, discounts, and delivery-related statuses. Sale item statuses include pending, packed, shipped, delivered, cancelled, and returned. The backend validates stock before a sale can commit stock. When the sale item is packed, shipped, or delivered, stock allocations are created from received purchase layers.

3.3.9 Billing Notes Page

The billing notes page was developed for customer receivable tracking. It groups eligible sales into billing notes and allows the user to track whether each line has been received. The backend controls eligibility so the same sale is not selected into more than one active billing note. The page also shows summary values, status filters, expected payment dates, actual payment dates, bank references, and linked credit note information.

3.3.10 Credit Notes Page

The credit notes page was developed for adjustments after a sale is cancelled or returned. The backend provides eligible sale lines, and the credit note stores customer information, sale reference, optional billing note link, line items, and total credited amount. This keeps the credit note connected to the original sale item while still preserving a readable credit document.

3.3.11 Products Page

The products page was developed as the main product catalog. It supports product creation, editing, disabling, deleting where allowed, SKUs, previous SKUs, names, category links, unit conversions, reorder levels, product pictures, supplier links, average cost information, and product transaction history. Products with transaction history can be disabled instead of removed, which protects old purchase, sale, and quotation records.

3.3.12 Categories Page

The categories page was developed to organize products in a parent-child tree. The full category tree is shown in one view because hierarchy is important for understanding the catalog. Categories are stored separately from products, and products reference categories through foreign keys while also keeping a category name snapshot for compatibility and readability.

3.3.13 Supplier Page

The supplier page was developed for maintaining supplier master data. It stores company name, locations, email addresses, telephone numbers, taxpayer ID, branches, shipping addresses, remarks, payment terms, and procurement contact details. Supplier records are reused in purchases, quotations, payment batches, and product-supplier sourcing links.

3.3.14 Customer Page

The customer page was developed for maintaining customer master data. It stores company name, locations, emails, telephone numbers, taxpayer ID, branches, shipping addresses, remarks, and billing/payment term information. Customer records are reused in sales, quotations, billing notes, and credit notes.

3.3.15 Login, User Access, and Settings Support

The frontend includes login-related behavior through the authentication context and API client. The backend provides Simple JWT login, refresh, and authen-

ticated profile endpoints. The frontend stores tokens in local storage or session storage depending on the login option, refreshes access tokens, and logs out when authentication expires. The backend-wide requirement for authentication can be turned on through the INVENTORY_REQUIRE_AUTH environment setting.

The system also includes user-access management. Authorized administrators can create users, assign roles, activate or deactivate accounts, and manage role permissions. The role system is based on Django groups and permissions. The implemented default roles are Admin, Manager, Sales, Purchasing, Accounting, and Viewer. These roles control which modules a user can view or update, such as products, purchases, sales, quotations, billing notes, payment batches, credit notes, customers, suppliers, user access, and activity logs.

3.4 System Architecture

The system uses a three-tier web architecture. React is responsible for the user interface, Django REST Framework provides the API and business rules, and MySQL stores the relational records. The frontend does not connect directly to the database. Every important action goes through the API.

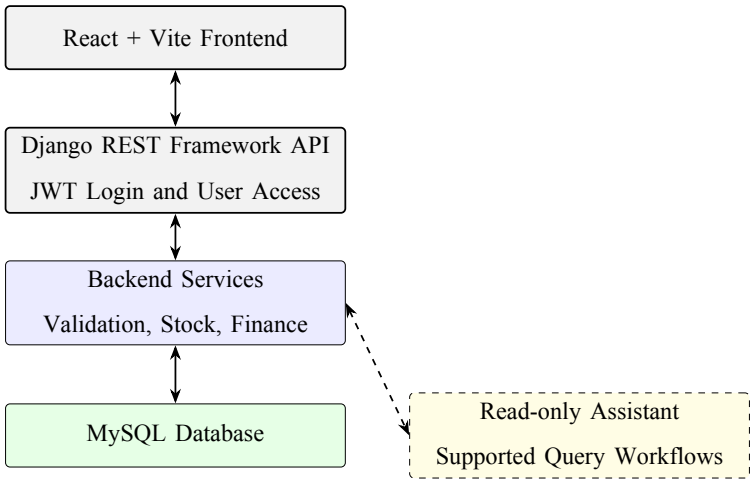

Figure 3.1 System architecture overview

Table 3.3 Architecture Components

Layer	Implementation	Main Responsibility
Frontend	React 18 and Vite	Builds the tabbed interface, forms, filters, tables, modals, and bilingual text display.
API	Django REST Framework	Receives requests, serializes data, applies filters, supports pagination, and returns JSON responses.
Business logic	Django service functions	Handles stock calculation, FIFO allocation, dashboard summaries, payable synchronization, eligibility checks, and assistant context.
Database	MySQL with Django ORM	Stores master data, transaction records, line items, documents, allocations, and finance records.
Authentication and access control	Simple JWT, Django groups, and Django permissions	Provides login, refresh token, authenticated profile support, role-based user access, and permission-gated API actions.

3.5 Frontend Methodology

The frontend was developed with React components, hooks, and helper files. Large pages are split into smaller sections, such as form details, line items, totals, directory tables, and detail modals. This made the code easier to manage because a purchase form, for example, has different concerns from a purchase history table.

The API client in frontend/src/api.js centralizes communication with the backend. It builds query strings, sends JSON or form-data requests, attaches bearer tokens, retries a request after token refresh, and dispatches an authentication-expired event when refresh fails. The frontend permission helper uses the profile returned from /api/auth/me/ to show or hide navigation tabs such as products, purchases, sales, user access, and activity log. The data-loading hooks keep full lookup data separate from paginated directory rows, so forms can still access reference data while history pages can use pagination.

The frontend also keeps a mock-data fallback for development and demonstration. This fallback is useful when the backend is not running, but it is not treated as the final authority. The backend remains the source of truth for stock validation, eligibility, and financial synchronization.

3.6 Backend and API Methodology

The backend was developed as a Django project with one main inventory application. Django REST Framework viewsets provide CRUD endpoints for products, categories, suppliers, customers, purchases, sales, quotations, billing notes, payment batches, and credit notes. Separate API views provide dashboard summaries, dashboard segments, lookup lists, product history, product stock layers, eligibility lists, and chat responses.

Table 3.4 Main API Endpoint Groups

Endpoint Group	Purpose
/api/products/, /api/ categories/	Product catalog and category tree management.
/api/suppliers/, /api/ customers/	Business partner master data management.
/api/purchases/, /api/sales/	Purchase and sales transaction workflows, including line items and documents.
/api/quotations/	Quotation records with normalized line items and supplier options.
/api/billing-notes/, /api/ payment-batches/, /api/ credit-notes/	Customer receivable, supplier payable, and credit note workflows.
/api/eligibility/...	Backend-controlled lists for billing-note sales, payment-batch purchases, and credit-note sale lines.
/api/dashboard/, /api/ dashboard/segment/	Dashboard summary and segmented operational reports.
/api/products/<id>/history/ and /stock-layers/	Product transaction history and received stock layer details.
/api/chat/	Read-only assistant answers for supported questions.
/api/ai-reports/	Generates printable HTML reports for a selected supplier, customer, or product with metrics, charts, and record tables.
/api/auth/login/, /refresh/, /me/	Login, token refresh, and authenticated user profile.
/api/admin/users/, /admin/ roles/	User account management, role assignment, and role permission management for authorized administrators.
/api/activity-logs/	Read-only activity history for login, create, update, and delete events, available only to users with activity-log access.

Pagination is opt-in. If a page parameter is not sent, list endpoints can still return a plain array for compatibility with existing frontend logic. When pagination is

requested, the response includes count, next, previous, page, page_size, total_pages, and results.

3.7 Database Design Methodology

The database was designed as a relational MySQL schema managed by Django migrations. Master data is separated from transaction data. For example, products, suppliers, customers, and categories are stored once and then referenced by purchases, sales, quotations, billing notes, payment batches, and credit notes.

The system also keeps snapshot fields inside transaction records. A sale item stores product name, SKU, quantity, price, and amount. A purchase stores supplier name and line item details. These snapshot values are important because old business documents should remain readable even if a product, supplier, or customer is edited later.

Table 3.5 Primary Database Model Groups

Group	Models
Master data	Category, Supplier, Customer, Product, ProductPicture, ProductUnitConversion, ProductSupplier
Purchasing	Purchase, PurchaseItem, PurchaseDocument
Sales	Sale, SaleItem, SaleItemAllocation, SaleDocument
Quotation	Quotation, QuotationItem, QuotationItemSupplier
Finance	BillingNote, BillingNoteLine, PaymentBatch, PaymentBatchLine, CreditNote, CreditNoteLine

3.8 Business Workflow Design

The main workflow starts from master data. Products belong to categories and are used in purchase, sale, and quotation line items. Purchases bring stock into the system only when purchase items are received. Sales commit stock only when sale items are packed, shipped, or delivered. This is why the system does not treat product stock as a manually edited field.

Quotations are used before purchase or sales records are created. Billing notes are created from eligible sales for customer receivable tracking. Payment batches are created from eligible purchases for supplier payable tracking. Credit notes are created

from eligible cancelled or returned sale lines.

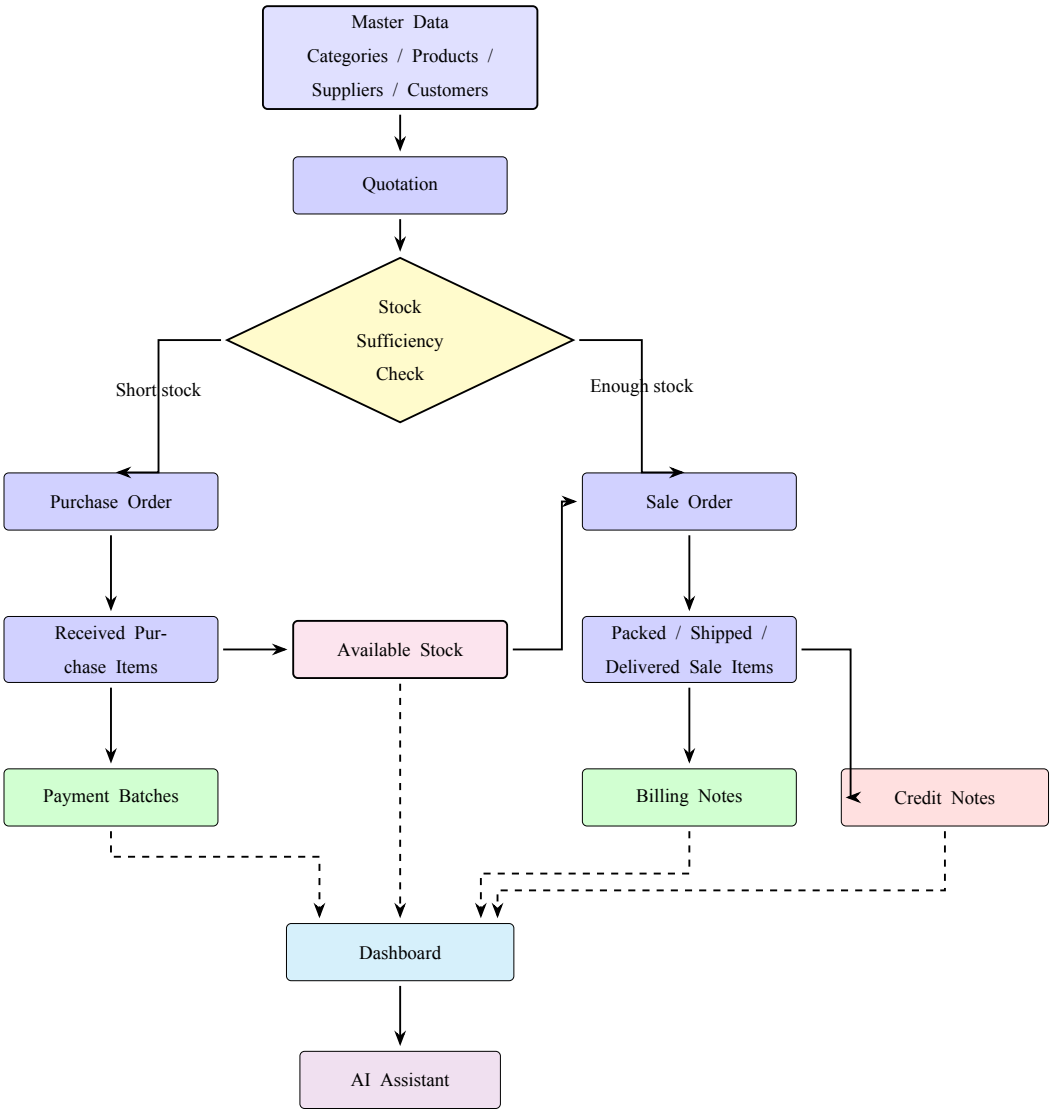

Figure 3.2 Business workflow design

3.9 Business Calculations and Rules

This section describes the main calculation logic that is implemented in the web application. Some totals are calculated in the frontend for immediate display, while the backend stores the submitted totals and applies the authoritative business rules for stock, eligibility, and payable synchronization.

3.9.1 Unit Conversion and Line Amounts

Products can have different purchase or sale units. The backend normalizes transaction quantities into the product base unit.

$$\text{Base Quantity} = \text{Quantity} \times \text{Conversion Factor}$$

Purchase and sale line amounts are calculated from quantity, unit price or unit cost, and any percentage discount chain. The frontend clamps each discount between 0% and 100%.

$$\text{Discounted Amount} = \text{Quantity} \times \text{Unit Price} \times \prod (1 - \frac{\text{Discount Percent}}{100})$$

For purchases, *Unit Price* means unit cost. For sales and quotations, it means sale price. Sales also apply the bill-level discount to the line amount when the sale total is calculated.

3.9.2 VAT Calculation

The purchase, sale, and quotation forms support VAT-included and VAT-not-included modes using a 7% VAT rate.

$$\text{VAT Rate} = 0.07$$

For VAT-included transactions:

$$\text{Total Before VAT} = \frac{\text{Item Total}}{1 + 0.07}$$

$$\text{VAT Amount} = \text{Item Total} - \text{Total Before VAT}$$

For VAT-not-included transactions:

$$\text{VAT Amount} = \text{Item Total} \times 0.07$$

$$\text{Grand Total} = \text{Item Total} + \text{VAT Amount}$$

3.9.3 Current Stock

The backend calculates product stock from transaction status instead of storing a manual stock number on the product.

$$\begin{aligned}\text{Current Stock} = \max(0,\ &\text{Received Purchase Base Quantity} \\ &- \text{Committed Sale Base Quantity})\end{aligned}$$

Received purchase quantity comes only from purchase items with received status. Committed sale quantity comes only from sale items with packed, shipped, or delivered status. Pending sales are shown separately as demand, but they do not reduce current stock until they reach a stock-deducting status.

3.9.4 Stock Validation and Allocation

Before a sale is allowed to commit stock, the backend compares requested committed quantity with available stock.

$$\text{Stock Issue exists if Requested Quantity} > \text{Available Quantity}$$

When stock is committed, the backend creates sale item allocations from received purchase layers. If the user does not choose stock layers manually, the backend uses first-in, first-out allocation. The received layers are ordered by received date, purchase transaction date, purchase creation date, and item ID.

$$\text{Allocation Quantity} = \min(\text{Remaining Sale Quantity}, \text{Layer Available Quantity})$$

Manual allocation is also supported, but the selected layers must match the product, must already be received, must have enough remaining quantity, and must add up exactly to the sale item base quantity.

3.9.5 Inventory Dashboard Calculations

The backend stock report calculates several useful inventory values for the dashboard and inventory page.

$$\text{Average Unit Cost} = \frac{\text{Received Purchase Value}}{\text{Received Purchase Units}}$$

$$\text{Average Daily Demand} = \frac{\text{Sales History Units}}{\text{Sales History Days}}$$

$$\text{Lead Time Demand} = \text{Average Daily Demand} \times \text{Average Lead Time}$$

$$\text{Safety Stock} = \text{Average Daily Demand} \times \text{Safety Stock Days}$$

$$\text{Calculated Reorder Level} = \lceil \text{Lead Time Demand} + \text{Safety Stock} \rceil$$

$$\text{Order-up-to Level} = \text{Reorder Level} + \lceil \text{Lead Time Demand} \rceil$$

$$\begin{aligned}\text{Recommended Restock} = \max(0,\ & \\ & \text{Order-up-to Level} + \text{Pending Sales} \\ & + \text{Oversold Units} \\ & - \text{Available Stock} - \text{Pending Purchases})\end{aligned}$$

$$\text{Stock Value} = \text{Available Stock} \times \text{Average Unit Cost}$$

The inventory page also uses supplier cost history to show supplier options. Supplier options are ranked by the most recent unit cost and order count, so the user can see a practical reorder source.

3.9.6 Purchase Payable and Payment Batch Calculations

The backend keeps the purchase grand total as the original document total, but it calculates a separate payable amount when purchase items are cancelled.

$$\text{Payable Total} = \text{Grand Total} \times \frac{\text{Non-cancelled Line Amounts}}{\text{All Line Amounts}}$$

This allows the system to keep the original purchase document readable while still showing the correct amount that should be paid to the supplier. Payment batch totals are calculated from their line amounts.

$$\text{Payment Batch Total} = \sum \text{Payment Batch Line Amount}$$

Unpaid payment batch lines are synchronized when the related purchase payable total changes. Paid lines are left unchanged because they represent a completed financial record.

3.9.7 Billing Note and Credit Note Calculations

Billing note totals come from selected eligible sales. Credit notes store line amounts from eligible cancelled or returned sale items. When a billing note has active credit notes, the net amount shown in the billing detail is reduced by the credit total.

$$\text{Billing Net Amount} = \text{Billing Note Total} - \text{Active Credit Note Total}$$

This makes the receivable amount clearer when a customer has a return or cancellation adjustment.

3.9.8 Cashflow and Net Position

The dashboard cashflow segment groups open receivables and open payables by due date. It also calculates the open net position.

$$\text{Open Net Position} = \text{Open Accounts Receivable} - \text{Open Accounts Payable}$$

Overdue accounts receivable are open billing notes with expected payment dates before today. Overdue accounts payable are open payment batches with planned payment dates before today.

3.9.9 Quotation Validity

Quotation validity is calculated from the quotation date and validity setting. If the validity mode is calendar days, the valid-until date is the quotation date plus the selected number of days. If the validity mode is business days, weekends are skipped.

The no-valid-date option stores no valid-until date.

3.10 Validation and Security Design

Validation is handled mainly by the backend. The frontend helps the user by showing form messages and previews, but direct API calls still need to be safe. The backend validates stock availability, active product usage, allocation rules, document eligibility, quotation validity dates, and finance document selection rules.

Security support is implemented with Simple JWT, Django groups, and Django model permissions. The backend exposes login, token refresh, and authenticated profile endpoints. The frontend stores and refreshes tokens, then attaches the access token to API requests. The authenticated profile includes the user's roles, permissions, and access flags for user management and activity logs. In local development, the backend can allow unauthenticated access depending on INVENTORY_REQUIRE_AUTH; when that setting is enabled, API access requires authentication and model-level permissions are checked for protected CRUD actions.

3.11 Testing Methodology

Testing was designed to check both technical correctness and user workflow understanding. The technical tests focused on backend behavior because the backend owns the most important business rules, including stock calculation, stock validation, finance eligibility, payable synchronization, and AI data preparation. Manual testing and questionnaires were added to check whether users could complete the main business workflow and understand the interface.

3.11.1 Automated Backend Validation Method

Automated backend validation was conducted with Django and Django REST Framework test cases. The tests used a temporary test database, created controlled records, called backend APIs or service functions, and compared the returned values with expected behavior. This method was used because backend rules must still be correct even if a user bypasses the frontend and calls the API directly.

Table 3.6 Testing Summary

Test Area	What It Checks
Pagination and filtering	Paginated responses, page size limits, search, status filters, date filters, and product or partner filters.
Product data	Product picture upload, active/disabled behavior, lookup data, stock filter, average cost, and product history.
Stock validation	Sales cannot commit more stock than available received purchase layers.
Stock allocation	FIFO allocation, manual allocation, and stock layer remaining quantity are handled correctly.
Reorder calculation	Fixed mock purchase and sale data is used to confirm available stock, average cost, lead time, daily demand, safety stock, reorder level, recommended restock, days until stockout, and stock value.
Finance eligibility	Billing notes, payment batches, and credit notes only use eligible records.
Financial synchronization	Purchase payable totals and payment batch line amounts stay consistent when purchases change.
User access control	Backend tests confirm that regular users cannot open administrator user endpoints, authorized administrators can create users and assign roles, and model permissions control protected API access when authentication enforcement is enabled.
Assistant scope	Supported questions return data-based answers, while unsupported questions are rejected as out of scope.
AI report output	Supplier, customer, and product report contexts are checked against backend data, and generated report HTML is sanitized and rendered with deterministic chart rows.

The preserved black-box test history contains a selected 30-case workflow suite for presentation. That selected suite covered product validation, quotation behavior, purchase workflow, sales stock validation, finance eligibility, credit notes, payment batches, payable synchronization, lookup endpoints, filters, and pagination. The current backend test coverage also includes additional focused tests for reorder calculation, AI Chat, AI Report, user access, permission enforcement, and activity logging. The test command used for the backend suite was:

backend/.venv/bin/python backend/manage.py test inventory

Table 3.7 Automated Backend Test Method

Method Item	Description
Test data	Controlled products, suppliers, customers, purchases, sales, quotations, billing notes, payment batches, and credit notes were created inside the test database.
Test action	The test cases called backend endpoints or backend service functions, similar to how the frontend sends requests during real use.
Expected output	The returned API data, status codes, calculated values, and validation errors were compared with the expected business rules.
Access-control checks	User-access tests checked that ordinary users cannot open administrator endpoints, administrators can create users and assign roles, permissions control protected API access, and activity-log access is restricted.
Reason for method	Automated tests are repeatable and suitable for checking exact rules such as stock validation, eligibility, and financial synchronization.

3.11.2 Calculation Validation Method

The calculation validation focused on the reorder point and inventory dashboard calculation. The method used fixed mock data instead of live random data so that the expected output could be manually calculated and compared with the backend result.

The test created one product, received purchase stock, committed sales, pending sales, pending purchases, and lead-time samples. After the records were created, the dashboard stock report calculation was executed. The output was checked against expected values such as available stock, average unit cost, average daily demand, safety stock, reorder level, recommended restock, days until stockout, and stock value.

Table 3.8 Calculation Validation Method

Method Item	Description
Input data	Fixed purchase and sale records were created for a known product so the expected stock calculation could be reproduced manually.
Test action	The backend dashboard stock report logic was called using the controlled test records.
Expected output	The backend should return the same reorder, restock, stock value, and stockout values as the manual calculation.
Reason for method	Reorder point output affects purchasing decisions, so the formula must be validated using traceable input data.

3.11.3 AI Chat Validation Method

AI Chat validation was designed to confirm that the assistant answered from backend data and stayed within the supported assistant scope. The tests used known database records and sent fixed chat questions to the backend chat function. The checked output included the answer text, presentation title, referenced records, and the model marker.

The method also tested unsupported questions. This was important because the assistant should not invent broad business reports or perform actions outside the current read-only assistant scope. When an API key setting was configured, the validation still checked that operational answers remained tied to backend summaries and selected records.

Table 3.9 AI Chat Validation Method

Method Item	Description
Input data	Known customer, sale, stock, billing, and payment records were prepared or selected from the backend context.
Test action	Fixed user questions were sent to AI Chat, such as customer activity, net position, low stock, and unsupported broad requests.
Expected output	Supported questions should return backend-based summaries, while unsupported questions should be rejected as outside scope.
Reason for method	AI Chat must help users read system data without inventing values or changing records.

3.11.4 AI Report Validation Method

AI Report validation checked whether report generation used the selected supplier, customer, or product records correctly. The method focused on the report context before and after the AI-written body was added. The backend prepared metrics, charts, record tables, and selected transaction references, then returned a self-contained printable HTML report.

The validation also checked report safety. Model-style HTML was simulated during testing so the system could verify that unsafe content was removed and that chart rows came from backend-prepared data. This makes the test repeatable and prevents the result from depending on changing AI output.

Table 3.10 AI Report Validation Method

Method Item	Description
Input data	A selected supplier, customer, or product was used with related purchases, sales, quotations, finance records, and stock data.
Test action	The report endpoint generated the report context and final HTML document for the selected scope.
Expected output	The report should contain correct selected-record references, summary metrics, backend chart rows, tables, a print button, and sanitized HTML.
Reason for method	AI Report may use AI for presentation, but the business values and charts must still come from reliable backend data.

3.11.5 Manual Scenario Test Method

Manual scenario testing was conducted to check whether users could complete the main business workflow in the system. The scenario was divided into parts that followed the intended workflow: quotation, purchase creation, purchase receiving, supplier payment, sale checking, status update, credit note, and billing note.

During the test, each participant followed the scenario steps and recorded the time and score for each part. Tester names were removed from the stored result file for privacy. This method was used because automated tests can prove that rules work, but they cannot show whether users can understand the screen flow and find the required actions.

Table 3.11 Manual Scenario Test Method

Method Item	Description
Input data	Testers used the prepared system data and followed scenario tasks covering the main transaction and finance workflow.
Test action	Testers completed each scenario part while completion time, task score, and comments were collected.
Expected output	Users should be able to complete the main workflow and identify where the system is easy or difficult to use.
Reason for method	Manual testing checks practical usability and workflow clarity from the user's point of view.

3.11.6 Questionnaire Method

After the scenario test, participants answered a questionnaire using a rating scale. The questionnaire measured their perception of usability, workflow clarity, system usefulness, and areas that may need improvement. Written comments were also reviewed to find repeated feedback themes.

The questionnaire was used as a supporting evaluation method. It did not replace automated validation, but it helped show whether the implemented system was understandable and useful to users after they tried the workflow.

Table 3.12 Questionnaire Method

Method Item	Description
Input data	Participants answered rating-scale questions and optional written feedback after completing the scenario test.
Test action	Scores were summarized, and comments were grouped into repeated themes such as easy areas, difficult areas, and visual clarity.
Expected output	The questionnaire should show general user perception and highlight improvement areas for the interface and workflow.
Reason for method	Questionnaire feedback adds user-centered evidence that cannot be captured by automated backend tests alone.

3.12 Tools and Technologies Used

The project used separate tools for frontend development, backend development, database storage, authentication, AI integration, styling, and testing. These tools were selected to support a full-stack web application with server-side business rules and

a React-based user interface.

Table 3.13 Tools and Technologies

Category	Tool or Technology	Use in This Project
Frontend framework	React 18	Builds the web application screens and reusable UI components.
Frontend build tool	Vite	Runs the local development server and production build process.
Backend framework	Django 5	Provides project structure, ORM, migrations, settings, and backend application logic.
API framework	Django REST Framework	Provides serializers, viewsets, API views, JSON responses, parsers, and API tests.
Database	MySQL	Stores relational inventory, transaction, and finance data.
Authentication and access control	Simple JWT and Django permissions	Provides access and refresh token authentication, authenticated profiles, user-role management, permission checks, and activity-log access control.
AI integration	OpenAI API with local fallback	Supports read-only assistant answers when configured, with local summary fallback when no API key is available.
Styling	CSS files by domain	Provides the compact inventory system interface.
Testing	Django TestCase and DRF APITestCase	Validates backend business rules and API behavior.

CHAPTER 4
RESULTS

This chapter explains the result of the implemented Inventory Management System. The result is based on the current codebase and the pages that are already implemented in the web application. The detailed development method and calculation logic were explained in Chapter 3, so this chapter focuses on what the completed system can do.

4.1 Completed System Overview

The final system is a full-stack web application for an SME trading business. It supports the flow from product setup to purchasing, selling, billing, supplier payment, credit note adjustment, inventory checking, dashboard reporting, and limited AI-assisted questions.

The main result is that the system connects business documents together instead of keeping them as separate records. A product can appear in purchases, sales, quotations, inventory reports, and product history. A sale can later become part of a billing note or a credit note. A purchase can later become part of a payment batch. This connection makes the system more useful than a simple product list.

4.2 Implemented Modules

4.3 How the System Looks in Practice

This section walks through selected pages from the finished application. Each screenshot is paired with a short explanation of what the page does and why it matters for day-to-day use. The goal is to give the reader a practical feel for the system before the detailed results are discussed.

Table 4.1 Implemented Workspace and Transaction Modules

Module	Implemented Result	Status
Dashboard	Shows stock, finance, cashflow, product movement, and order coverage summaries.	Completed
Inventory	Shows backend-calculated stock rows, reorder need, supplier options, product details, and quick purchase-order preparation.	Completed
AI Chat	Answers supported read-only questions about stock, fulfillment, partner summaries, AR/AP exceptions, and document references.	Completed
AI Report	Generates printable business reports for a selected supplier, customer, or product with metrics, charts, record tables, and AI analysis.	Completed
Purchases	Supports purchase records, line items, receiving status, documents, discounts, VAT totals, payable total, and payment batch links.	Completed
Payment Batches	Groups eligible purchases for supplier payment and tracks paid or unpaid lines.	Completed
Quotations	Supports quotation records, line items, supplier options, validity settings, VAT totals, and conversion to purchase or sale flow.	Completed
Sales	Supports sales records, stock validation, stock allocation, delivery status, documents, VAT totals, billing links, and credit links.	Completed

Table 4.2 Implemented Finance and Master Data Modules

Module	Implemented Result	Status
Billing Notes	Groups eligible sales for customer receivable tracking and tracks received or outstanding lines.	Completed
Credit Notes	Creates credit notes from eligible cancelled or returned sale items.	Completed
Products	Supports product CRUD, SKUs, pictures, unit conversions, reorder levels, active status, supplier links, and product history.	Completed
Categories	Supports category records with parent-child hierarchy.	Completed
Suppliers	Supports supplier profile, contact, branch, shipping address, procurement contact, and payment term data.	Completed
Customers	Supports customer profile, contact, branch, shipping address, and billing term data.	Completed

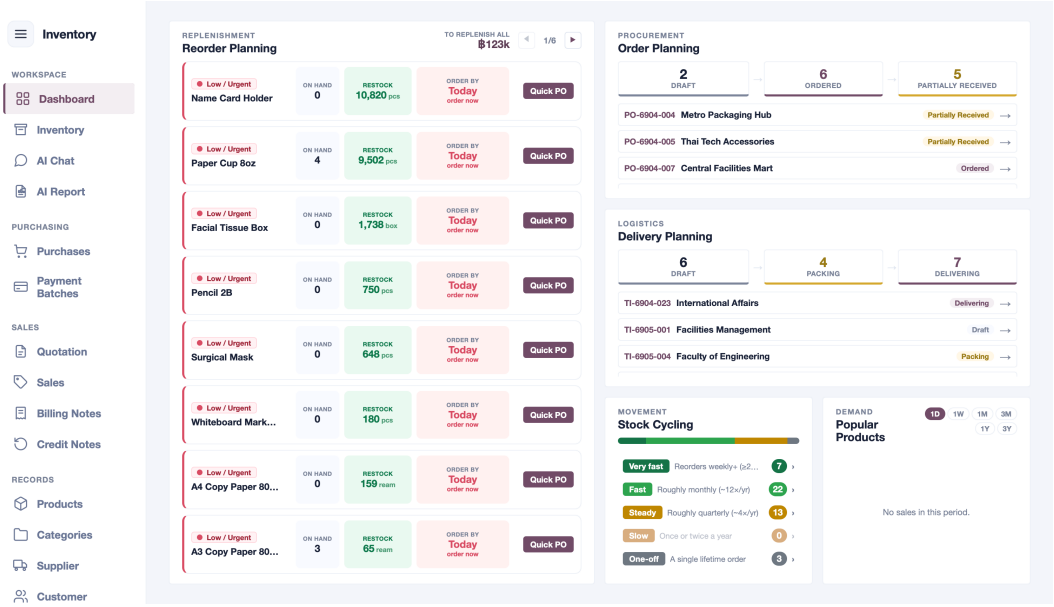

Figure 4.1 Dashboard page

Figure 4.1 is the first thing the user sees after logging in. It pulls together stock health, finance totals, cashflow projections, and order coverage into a single view, so the user can spot problems without jumping between pages.

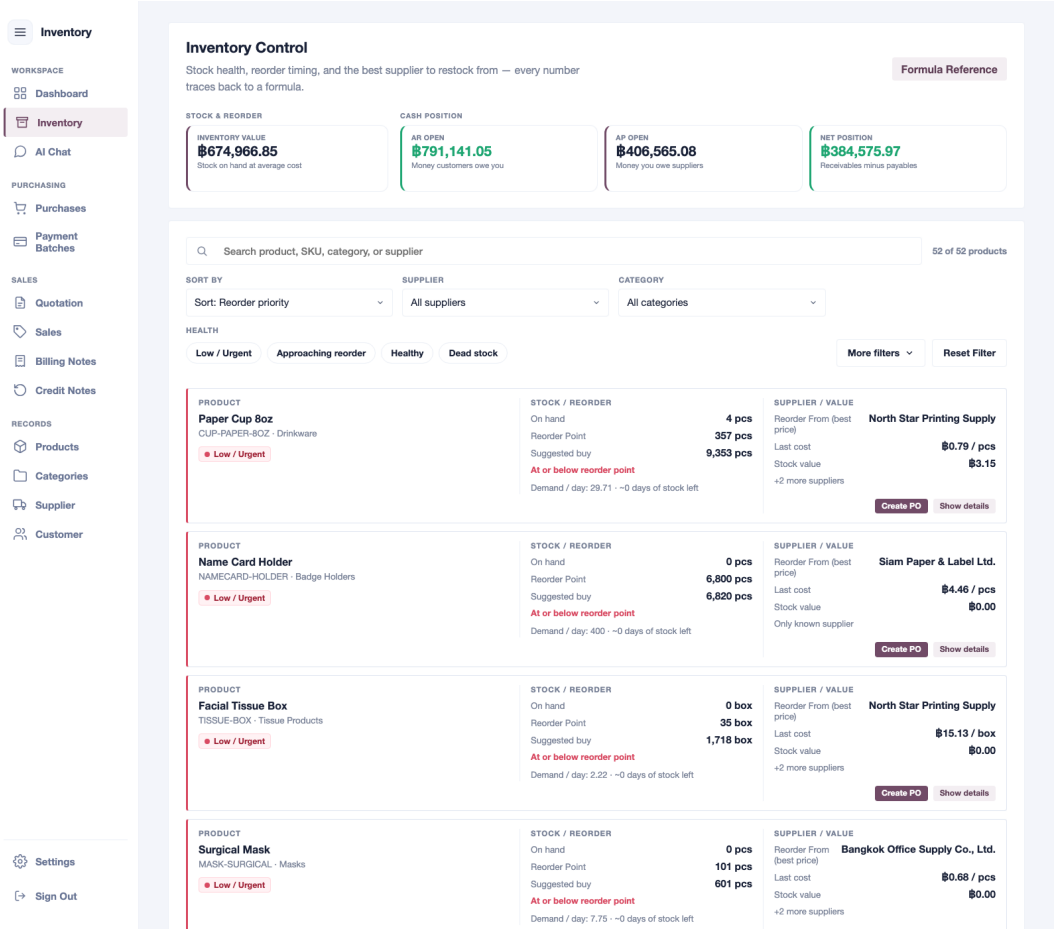

Figure 4.2 Inventory page

Figure 4.2 is the daily stock-checking workspace. It shows how many units are on hand, which products need reordering, which suppliers offer the best price, and how quickly each item is selling. All of this is calculated by the backend so the numbers stay reliable.

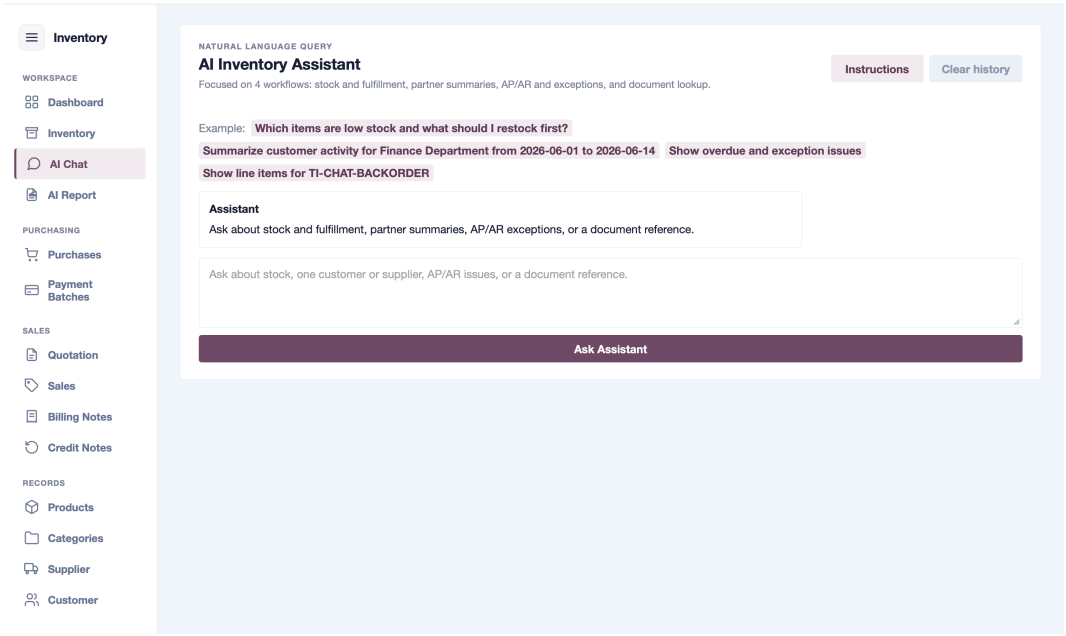

Figure 4.3 AI chat assistant page

Figure 4.3 shows the AI chat page where users can type natural-language questions about stock, partners, receivables, payables, or document references. The assistant reads real data from the backend and returns a focused answer, so users can get quick insights without navigating through multiple pages or filtering tables manually.

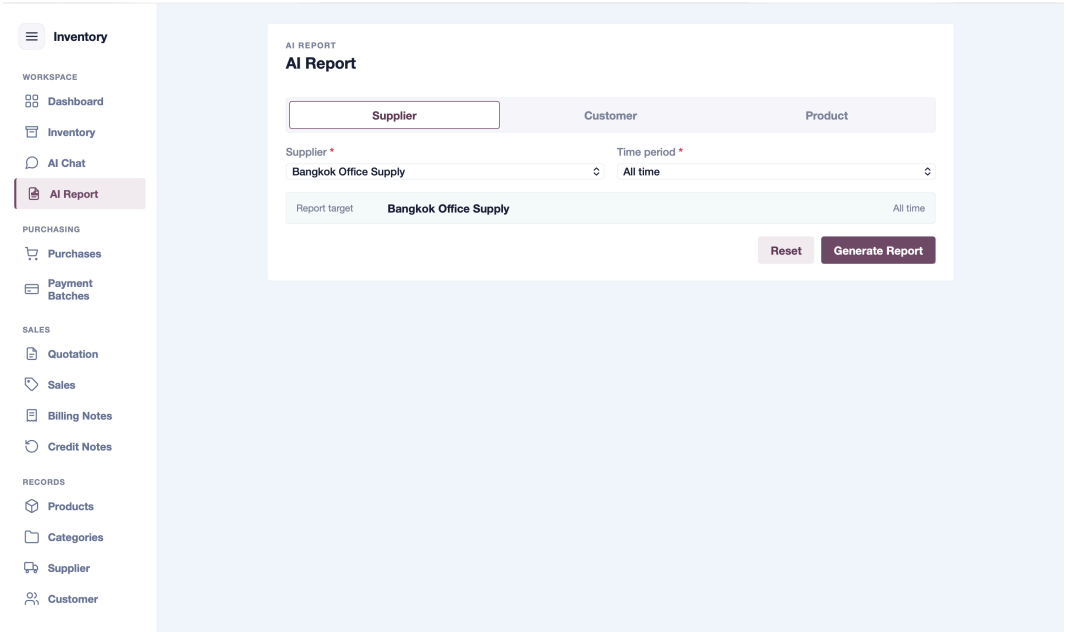

Figure 4.4 AI report generation page

Figure 4.4 shows the AI report page. The user picks a scope (supplier, cus-

tomer, or product), selects the specific record, and optionally narrows the date range. After clicking generate, a print-ready report opens in a new tab with summary metrics, trend charts, record tables, and an AI-written business analysis. This gives the user a shareable document without having to export data and build the report manually.

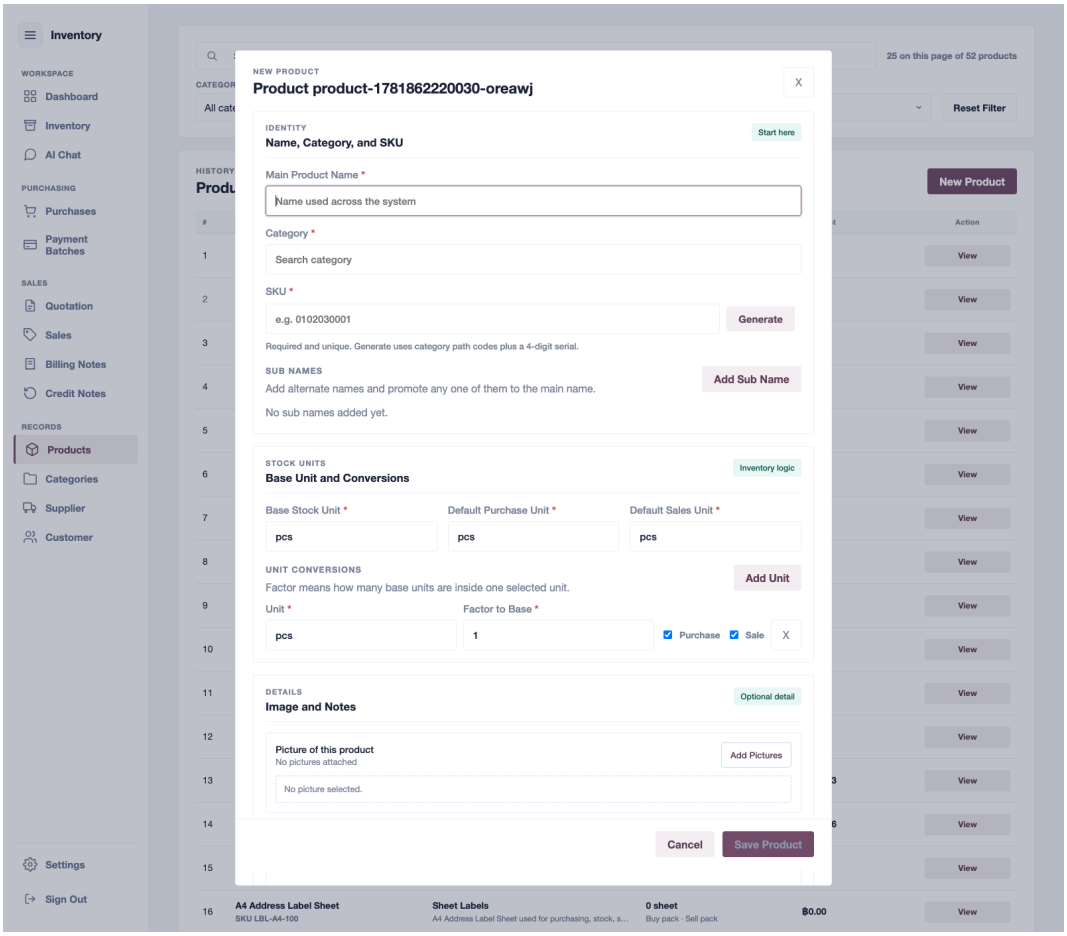

Figure 4.5 Product management form

Figure 4.5 shows the product form where the user manages item details. Everything from SKUs and unit conversions to reorder levels, product pictures, and linked suppliers is handled in one place.

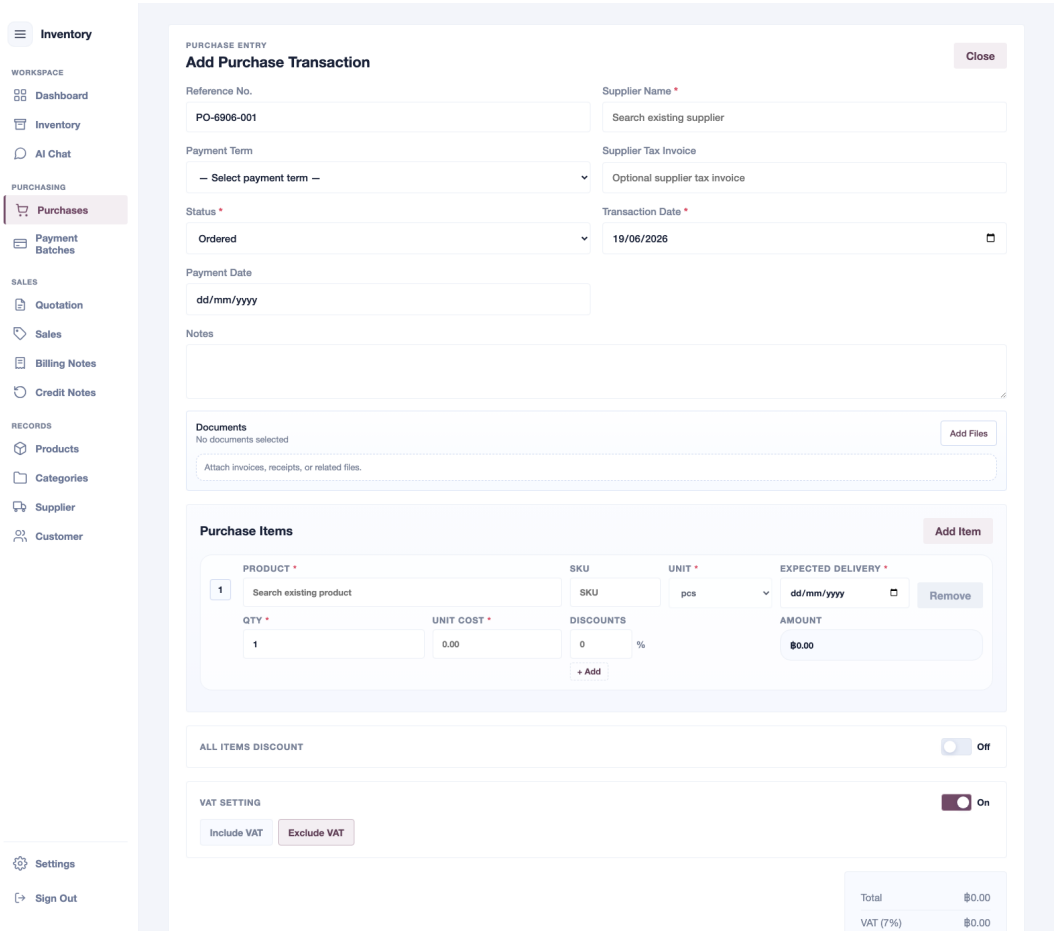

Figure 4.6 Purchase workflow

Figure 4.6 walks through how a purchase order is entered. The page lets the user add line items, track receiving status, set expected delivery dates, and see the running totals, all of which feed into stock levels and supplier payable tracking.

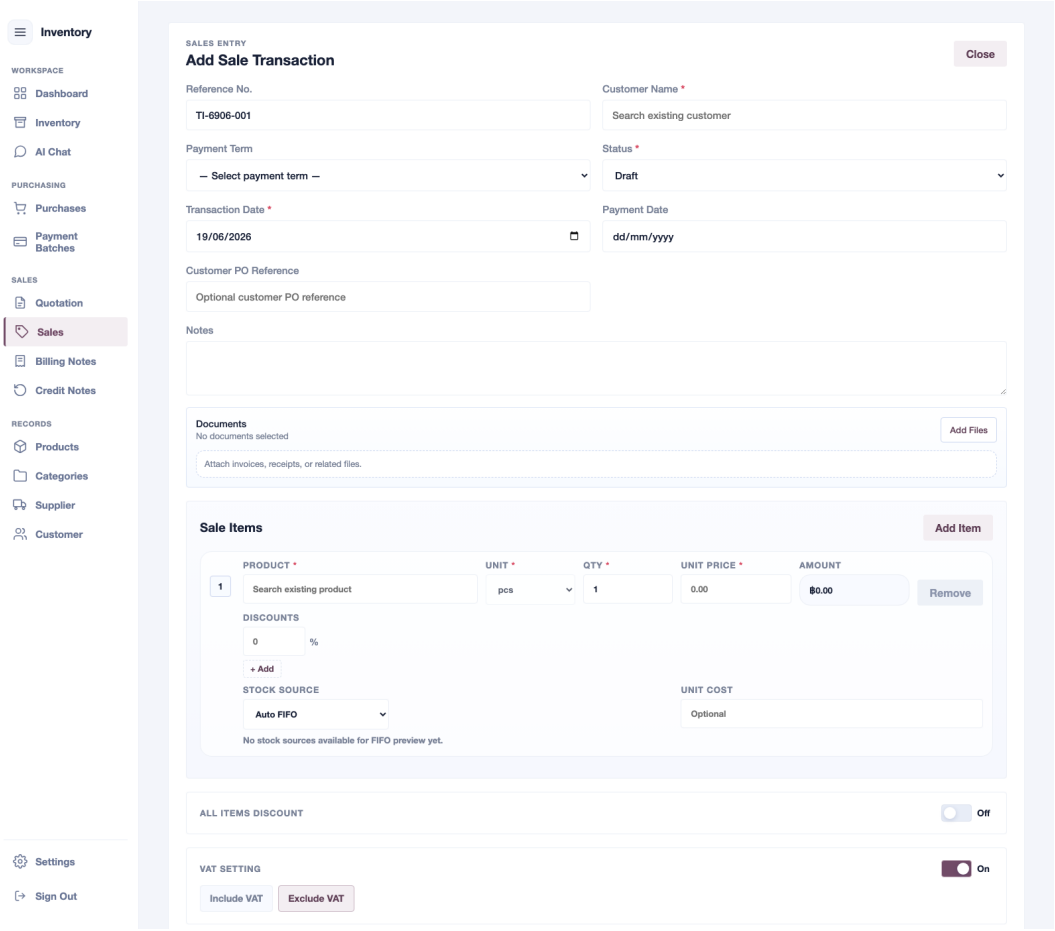

Figure 4.7 Sales workflow

Figure 4.7 shows how a customer sale is prepared and tracked. The user fills in customer references, adds stock-linked line items, and monitors delivery statuses. Once items are packed or shipped, the system commits stock and creates allocation records automatically.

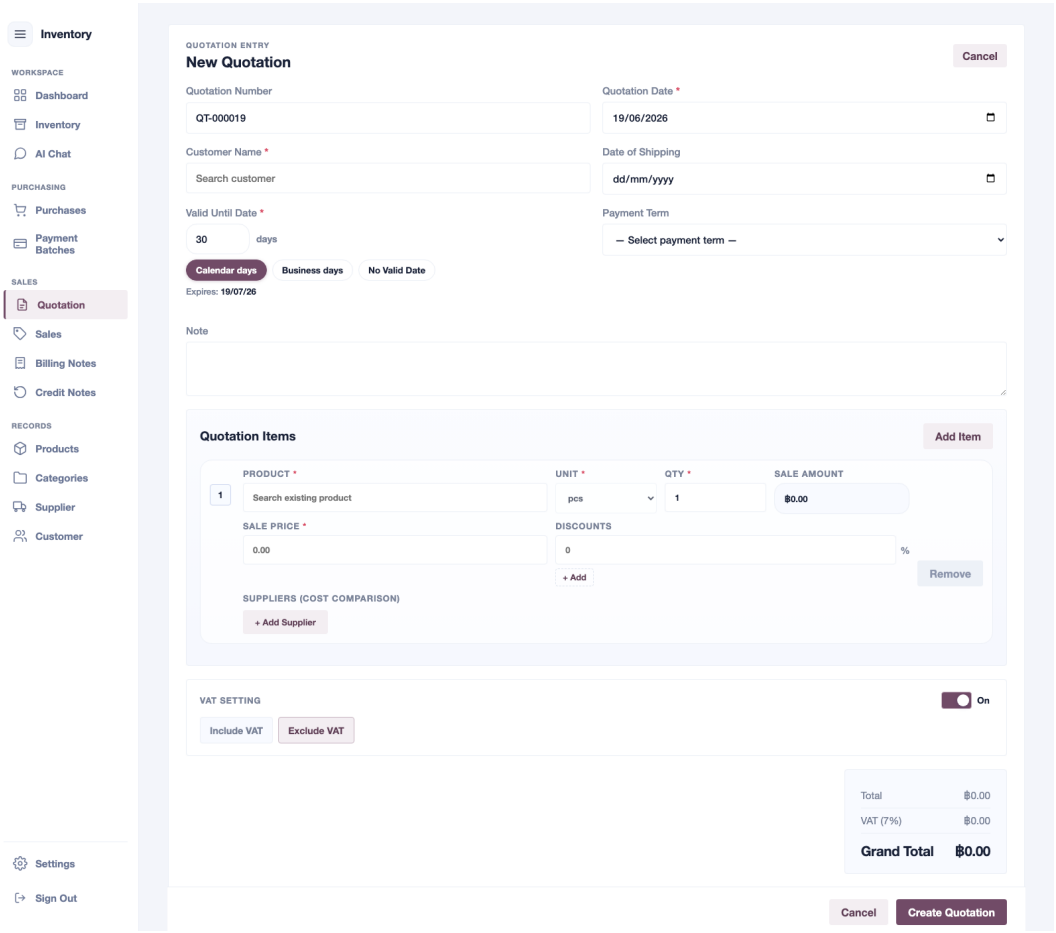

Figure 4.8 Quotation workflow

Figure 4.8 shows a quotation before it turns into a real purchase or sale. The user can compare supplier pricing, set validity dates, review line totals, and then convert the quotation into one or more purchase orders or a sales record.

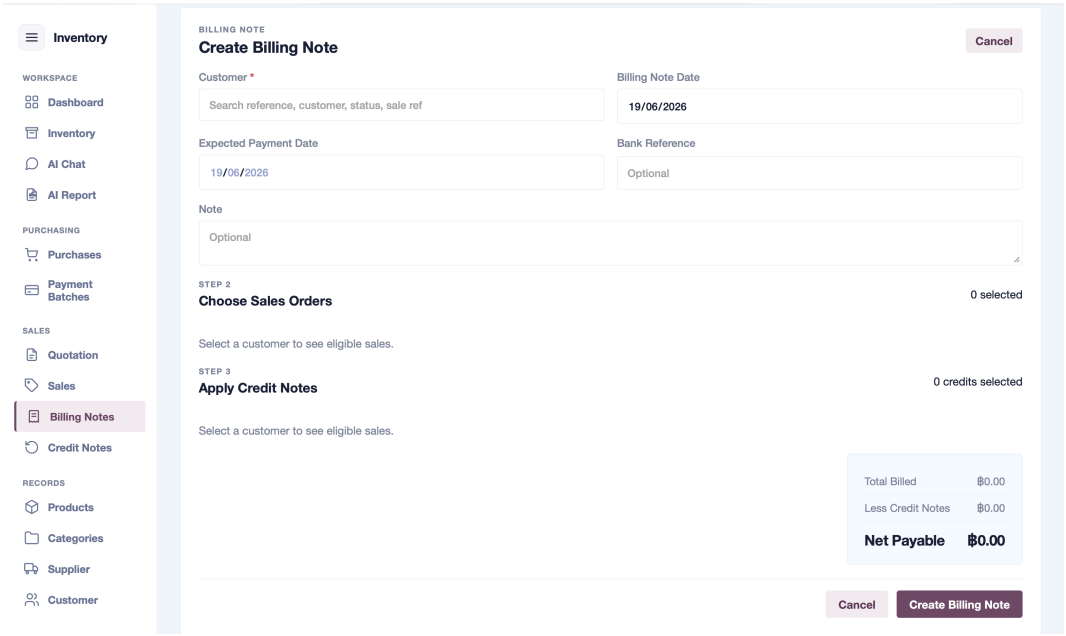

Figure 4.9 Billing note workflow

Figure 4.9 shows how eligible sales are grouped into a billing note for customer invoicing. The user can see which sales are included, track whether payment has been received, and review expected and actual payment dates.

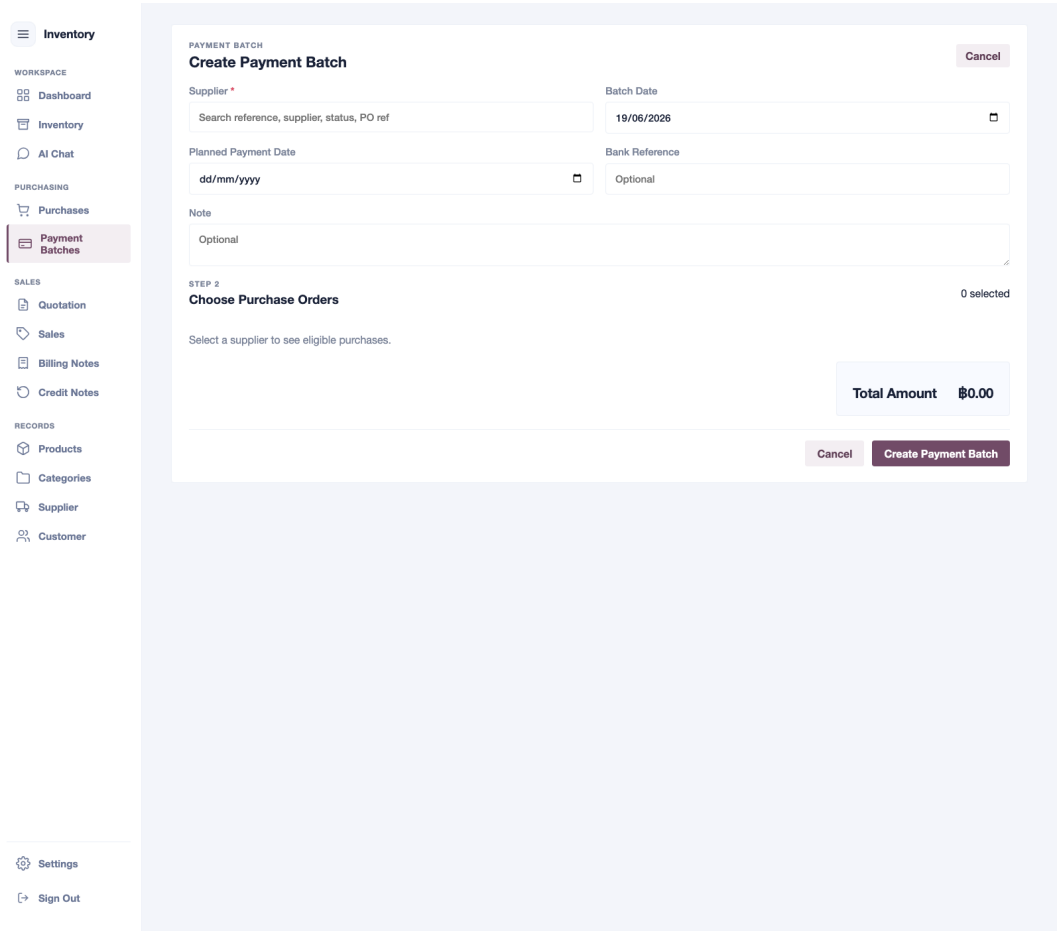

Figure 4.10 Payment batch workflow

Figure 4.10 shows the payment batch page for supplier payables. Eligible purchases are grouped into one batch, and the user can mark individual lines as paid and record bank references, keeping supplier payment tracking in one place.

4.4 Results by Workflow

4.4.1 Master Data Results

The master data pages were completed for products, categories, suppliers, and customers. Products can store SKUs, previous SKUs, names, categories, stock base units, default purchase and sales units, reorder levels, pictures, supplier links, and active status. Categories support hierarchy through parent categories. Suppliers and customers store contact, branch, shipping address, taxpayer, remark, and payment term information.

This result is important because the transaction pages depend on these records. Purchases need suppliers and products. Sales need customers and products. Quotations can use products, suppliers, and customers. The master data pages therefore work as the foundation of the system.

4.4.2 Purchasing and Stock Results

The purchase workflow was completed with purchase headers, purchase line items, item receiving status, expected delivery date, received date, uploaded documents, VAT fields, discounts, grand total, and payable total. A received purchase item increases available stock, while pending and cancelled purchase items do not.

The stock result is calculated by the backend. The inventory and dashboard pages show current stock, pending purchase units, delayed purchase units, pending sales units, recommended restock, stock value, and supplier options. This makes the inventory page useful for daily checking because the user does not need to manually combine purchase and sales records.

4.4.3 Sales Results

The sales workflow was completed with sale headers, customer information, customer purchase order reference, sale line items, delivery statuses, uploaded documents, VAT fields, discounts, billing note links, credit note links, and source quotation links.

The most important result in the sales module is server-side stock validation. A sale can stay as a draft even if there is not enough stock, but it cannot commit unavailable stock when the sale item becomes packed, shipped, or delivered. When stock is committed, the backend creates allocation records from received purchase layers. This gives better traceability between stock received from suppliers and stock sold to customers.

4.4.4 Quotation Results

The quotation workflow was completed with quotation date, validity settings, customer and supplier references, shipping date, payment terms, VAT mode,

total values, line items, and supplier options for each line. Quotation lines are stored in relational tables, not as one large JSON field. The frontend can also start purchase or sale creation from a quotation.

4.4.5 Finance Results

The finance workflow was completed through billing notes, payment batches, and credit notes. Billing notes group eligible sales for customer receivables. Payment batches group eligible purchases for supplier payables. Credit notes are created from eligible cancelled or returned sale items.

The backend eligibility endpoints are one of the useful results of this implementation. They reduce the chance of selecting the same sale into multiple active billing notes, selecting the same purchase into multiple active payment batches, or crediting the same returned/cancelled sale item more than once.

4.4.6 Dashboard and Inventory Results

The dashboard and inventory pages show the business status without loading every transaction into the frontend. The backend prepares stock report rows, finance summaries, cashflow buckets, product movement, popular products, and order coverage data. This result makes the interface faster and easier to use because the frontend receives summary data that is already prepared for display.

4.4.7 AI Assistant Results

The AI assistant was implemented as a limited read-only assistant. It can answer supported questions about low stock, restock scope, net position, order coverage, customer summary, supplier summary, overdue or exception issues, and line items for a known reference. The tests also show that questions outside these supported workflows are rejected as outside the current assistant scope.

4.4.8 AI Report Results

The AI report page was completed with scope selection, record selection, optional custom date range, and report generation. The backend collects purchases,

sales, quotations, billing notes, payment batches, credit notes, and stock data for the selected supplier, customer, or product. It calculates summary metrics such as purchase totals, sales totals, open payables, open receivables, available stock, gross margin, and margin rate. It also builds trend charts, top product or partner charts, and record tables. When an OpenAI API key is configured, the backend sends the data context to the AI model for an executive summary and business analysis. When no API key is available, the backend builds a local report from the same data. The generated report opens in a new browser tab as a self-contained, print-ready HTML document with bilingual support.

4.4.9 User Access and Activity Log Results

The user-access function was completed with login, token refresh, authenticated user profile, role assignment, and permission management. Authorized administrators can create users, update account status, assign roles, and manage the permissions inside each role. The frontend uses the authenticated profile to show only the pages that the user is allowed to access. The backend also stores activity logs for important actions such as login, create, update, and delete events, and the activity-log page is only available to users with the correct access.

4.5 Feature Evidence Summary

4.6 Key Findings

The first key finding is that inventory stock should be derived from transactions, not typed manually into the product record. In this system, stock depends on received purchase items and committed sales. This makes stock more reliable because it follows the actual business documents.

The second key finding is that finance documents need backend eligibility rules. Billing notes, payment batches, and credit notes are easy to duplicate by mistake if the frontend is the only place checking them. Moving the rules to the backend makes the workflow safer.

The third key finding is that historical snapshot fields are useful. Even when products, suppliers, and customers are stored as master data, old transactions still need

Table 4.3 Evidence of Implemented Features

Feature	Evidence in the Implemented System
REST API communication	The frontend API client calls endpoints for dashboard, lookups, products, purchases, sales, quotations, billing notes, payment batches, credit notes, authentication, and chat.
JWT authentication and user access	The backend provides login, refresh, profile, user-management, role-management, and activity-log endpoints. The frontend stores and refreshes access tokens, shows permission-based navigation, and provides a user-access page for authorized administrators.
Backend stock validation	The sale workflow calls backend validation before stock-deducting sale statuses are committed.
FIFO and manual allocation	The backend creates sale allocations from received purchase layers and validates manual layer selections.
Finance eligibility	Backend endpoints return only eligible sales for billing notes, eligible purchases for payment batches, and eligible sale lines for credit notes.
Normalized quotation items	Quotation lines and supplier options are stored in relational tables while the API still returns a simple items array to the frontend.
Backend dashboard summaries	The dashboard and inventory pages use backend-prepared summary data instead of calculating all operational metrics from full transaction lists in the browser.
Assistant scope control	The chat service answers only supported read-only workflows and rejects unsupported requests as outside scope.
AI report generation	The AI report endpoint collects transaction data for a selected record, calculates metrics and charts, and returns a printable HTML report with optional AI-written analysis.

to keep names, SKUs, prices, and totals so that the document remains understandable later.

4.7 Discussion of Objective Achievement

The first objective was to record purchase and sales transactions with process status and attachment support. This objective is achieved because purchases and sales both support line items, statuses, document uploads, and transaction detail views.

The second objective was to provide inventory stock information and calculation support. This objective is achieved through backend stock calculations, inventory page stock rows, dashboard summaries, product history, stock layers, reorder information, and recommended restock values.

The third objective was to integrate an AI chatbot connected to backend data. This objective is achieved within the current implemented scope. The assistant can answer supported read-only operational questions from backend data, but it is not a general-purpose chatbot and it does not change system records.

4.8 Testing and Evaluation Results

The system was evaluated using three types of evidence: automated backend validation, manual scenario testing, and user questionnaire results. Automated tests were used for business rules that must be exact, such as stock calculation, eligibility, and AI output validation. Scenario testing and questionnaires were used to evaluate how users performed the main workflow and how they perceived the system.

4.8.1 Automated Backend Validation

The selected backend validation history contained 30 black-box aligned automated test cases. These test cases were selected for presentation because they are easy to explain as user-facing workflow tests. They covered valid and invalid workflows for product rules, quotation behavior, purchases, sale stock validation, billing note eligibility, payment batch eligibility, credit notes, and payable synchronization. The latest recorded run passed all 30 cases.

Additional focused validation was performed for the reorder point calculation, AI Chat, AI Report, user access, permission enforcement, and activity logging. The focused AI and calculation validation run contained 5 tests and all 5 passed. A separate user-access and activity-log validation run contained 6 tests and all 6 passed. The reorder point test confirmed that fixed purchase and sale data produced the expected available stock, average cost, demand, safety stock, reorder level, recommended restock, days until stockout, and stock value. The AI tests confirmed that AI Chat uses backend facts, AI Report receives correct backend context, report charts are rendered from backend chart rows, and unsafe model HTML is removed before the report is returned. The access tests confirmed that ordinary users cannot open administrator user endpoints, administrators can create users and assign roles, protected model APIs enforce permissions when authentication enforcement is enabled, and activity-log access is restricted.

Table 4.4 Automated Validation Result Summary

Test Group	Result	Main Evidence
Selected backend workflow tests	30/30 passed	Product rules, quotation behavior, purchase workflow, sale stock validation, finance eligibility, credit notes, payment batches, and payable synchronization passed.
Reorder point calculation	Passed	The backend returned reorder level 48, recommended restock 18, days until stockout 15, and stock value 300 from fixed mock data.
AI Chat validation	Passed	AI Chat returned backend-based summaries for supported questions and rejected unsupported broad questions.
AI Report validation	Passed	Supplier, customer, and product reports used backend context, returned printable HTML, replaced poor model chart HTML with backend chart rows, and sanitized unsafe HTML.
User access and activity log validation	6/6 passed	Regular users were blocked from administrator endpoints, administrators could create users and assign roles, model permissions controlled protected access, and activity-log access was restricted.

4.8.2 Calculation Validation Detail

The calculation validation focused on the reorder point and recommended restock calculation because this is one of the most important inventory decisions in the system. The test did not use random data. Instead, it created fixed purchase and

sale records in a temporary Django test database so that the expected result could be calculated manually.

The test first created one product, two received purchase records, one pending purchase record, two committed sales, and one pending sale. Then the test called the dashboard API, which runs the same backend stock report calculation used by the real dashboard and inventory page. The backend output was compared with the manually calculated expected values.

Table 4.5 Calculation Validation Input

Input Item	Value	Reason for Test
Product	TEST-PEN	A single controlled product makes the stock row easy to identify.
Received purchase units	100 pcs	Used as the stock added into the system.
Received purchase value	500	Used to calculate average unit cost.
Committed sale units	40 pcs	Used as stock already allocated to fulfilled sales.
Pending sale units	20 pcs	Used as future demand that should affect restock recommendation.
Pending purchase units	10 pcs	Used as incoming stock that should reduce restock recommendation.
Lead time samples	4 days and 6 days	Used to calculate average lead time of 5 days.
Sales history	40 pcs over 10 days	Used to calculate average daily demand of 4 pcs per day.

This result shows that the backend formula matched the manually expected values. The important point is that the dashboard did not only display a number; it returned values that could be reproduced from the input data.

4.8.3 AI Chat Validation Detail

The AI Chat validation checked whether the chat page answers from backend data instead of inventing business values. The test was performed by calling the backend chat function with fixed questions and known data. The important output fields were the answer text, the presentation title, the selected record references, and the model marker.

Table 4.6 Calculation Validation Output

Output Item	Expected Value	Backend Value	Result
Available stock	60	60	Passed
Average unit cost	5	5	Passed
Average lead time	5 days	5 days	Passed
Average daily demand	4 pcs/day	4 pcs/day	Passed
Safety stock	28	28	Passed
Reorder level	48	48	Passed
Recommended restock	18	18	Passed
Days until stockout	15 days	15 days	Passed
Stock value	300	300	Passed

In this implementation, AI Chat returns a deterministic backend summary for supported operational questions. This means the answer is generated from the current database context and is marked as local-summary. The test also checked that unsupported broad questions are rejected instead of being answered freely.

Table 4.7 AI Chat Validation Input and Output

Test Question	Input Data Used	Expected Output Checked	Result
Customer activity summary	Customer with one known sale worth 275	Chat title shows the selected customer, sales count is 1, sales total is 275, and the sale record target points to the created sale.	Passed
Net position question	Existing billing note and payment batch data	Answer includes backend AR, AP, and net position values with presentation title ``Net position''.	Passed
Low stock question	Product stock report rows from backend	Answer lists low-stock products and restock priorities from backend stock calculations.	Passed
Unsupported quotation summary	A broad request outside the supported assistant scope	Assistant returns ``Outside current assistant scope'' and does not invent a quotation report.	Passed
Configured key condition	API key setting temporarily enabled during the test	Chat still returns local-summary, proving that operational chat answers remain backend-controlled.	Passed

This validation shows that AI Chat works as a controlled assistant. It

can help users find operational information faster, but the important numbers still come from backend records and backend calculations.

4.8.4 AI Report Validation Detail

The AI Report validation checked whether supplier, customer, and product reports are generated from the correct selected records. The test also checked that the final report is printable HTML, that chart rows come from backend data, and that unsafe model HTML is removed.

The report test used a controlled supplier and purchase record. The backend built a report context containing the supplier name, purchase reference, purchase amount, metrics, chart rows, and tables. With the API key setting configured, the model response was simulated so that the test could check the system behavior without depending on internet availability or changing AI output. This is important because automated tests must be repeatable.

Table 4.8 AI Report Validation Input

Input Item	Value	Purpose
Report scope	Supplier report	Confirms that the selected report type controls which records are collected.
Selected supplier	Configured Key Supplier	Confirms that the report context uses the selected supplier.
Product	Configured Key Product	Confirms that line-item data is included in the report context.
Purchase reference	PO-AI-KEY-001	Confirms that the report includes the expected transaction record.
Purchase amount	THB 450.00	Confirms that report totals and table values match backend data.
Model output	Simulated HTML body	Confirms how the system wraps model-style output into a final report document.

This validation shows that AI Report can use AI for presentation, but the selected records, metrics, charts, and tables are still prepared by the backend. The report therefore remains tied to system data rather than free-form model guessing.

Table 4.9 AI Report Validation Output

Output Checked	Expected Result	Result
Report context	Contains Configured Key Supplier, PO-AI-KEY-001, and THB 450.00.	Passed
Printable document	Final HTML contains document structure and a print button.	Passed
AI model body handling	Simulated model summary remains in the final report after wrapping.	Passed
Chart rendering	Poor model chart labels are replaced with backend chart row labels.	Passed
HTML safety	Script tags, unsafe event handlers, iframes, and JavaScript links are removed.	Passed
Current-data report check	Supplier, customer, and product reports generated from current database records and contained expected references and totals.	Passed

4.8.5 Manual Scenario Test

The manual scenario test asked users to complete the main business flow from quotation to purchase, purchase receiving, payment batch, sale, credit note, and billing note. Ten testers completed the scenario test. Tester names were removed from the stored result file and replaced with anonymous labels for privacy.

Table 4.10 Scenario Test Participant Summary

Item	Result
Number of participants	10 testers
Age range	19 to 68 years old
Average age	43.5 years old
Roles represented	SME-related users and general evaluators, including owner, manager, purchasing/sales staff, executives, and students.
Scenario parts	Part A to Part H, covering quotation, purchase, receiving, payment, sale, credit note, and billing note workflow.

The average scenario score across all parts was 9.21 out of 10. This suggests that testers were generally able to complete the main workflow successfully. The lower relative scores in Part B and Part F match the written comments, where some users mentioned that quotation-to-purchase flow, sale form details, or status buttons needed clearer visual guidance.

4.8.6 Questionnaire Result

After the scenario test, testers answered a 15-item questionnaire using a 1 to 5 rating scale. The average questionnaire score across all testers was 4.58 out of 5. This indicates a positive overall response to the system.

4.8.7 User Feedback Summary

The user comments showed several repeated themes. Positive feedback mentioned that purchase order status was understandable, quotation-to-purchase flow was useful, credit note creation could be simple, and connected data reduced the need for repeated entry. Improvement feedback focused mainly on visual clarity and consistency.

Overall, the evaluation results show that the implemented system can support the intended business workflow, but the interface can still be improved by making important buttons, statuses, and financial actions easier to find.

4.9 Limitations Found During Development

The system is functional, but some limitations remain:

- Backend-wide authentication is configurable, so the final deployment setting must be checked before production use.
- User access is implemented with role and permission management, but production deployment should still review the final staff roles, permission assignments, and administrator accounts before real company use.
- The frontend still keeps mock-data fallback behavior for demonstration, so final evaluation should use backend-connected data.
- Large-scale production performance testing with many users and a very large database was not completed in the current codebase.
- User testing was conducted with 10 participants, so the result is useful for project evaluation but not enough to represent all possible SME users.

Table 4.11 Average Scenario Test Time and Score

Part	Average Time	Average Score	Interpretation
Part A	151.9 seconds	9.0/10	Initial quotation workflow took the longest because it starts the process and requires more data entry.
Part B	71.8 seconds	8.9/10	Purchase creation from quotation was generally successful, but some users needed time to understand the flow.
Part C	29.7 seconds	9.5/10	Receiving purchase items was one of the faster and clearer tasks.
Part D	65.1 seconds	9.5/10	Payment-related workflow was completed with high scores.
Part E	40.4 seconds	9.2/10	Sale-related checking was completed successfully, although some users needed to read more information.
Part F	43.1 seconds	8.9/10	Status-related actions were completed, but comments showed that status buttons should be clearer.
Part G	7.2 seconds	9.3/10	The shortest part, showing that the required action was quick once found.
Part H	49.1 seconds	9.4/10	Final billing or finance-related workflow was completed with high scores.

Table 4.12 Questionnaire Result Summary

Evaluation Item	Result
Number of questionnaire responses	10 responses
Questionnaire items	15 items
Rating scale	1 to 5
Average questionnaire score	4.58/5
General result	Users gave mostly high scores, showing that the system was understandable and usable after completing the scenario.

Table 4.13 User Feedback Themes

Theme	Summary of Feedback
Easy areas	Users found purchase status, quotation creation, quotation-to-purchase flow, and connected data flow useful and understandable.
Difficult areas	Some users had difficulty finding buttons, especially save buttons, detail buttons, status buttons, and finance-related options such as billing notes or credit notes.
Visual clarity	Several comments suggested stronger colors, clearer status indicators, and more obvious emphasis for important actions or warning values.
Workflow consistency	Some users felt that clickable rows and save-button placement should be more consistent across pages.
Information density	Some users felt that certain pages, such as dashboard charts or cost information sections, contained many details and should be simplified or visually reorganized.

CHAPTER 5
CONCLUSION

5.1 Brief Project Overview

The Inventory Management System was developed as a web application for SME trading businesses that buy products from suppliers and resell them to customers. The system covers the main daily work of this type of business, including product records, suppliers, customers, purchases, sales, quotations, billing notes, payment batches, credit notes, inventory checking, dashboard summaries, limited AI-assisted questions, and printable AI-generated reports.

The project was built with a React and Vite frontend, a Django REST Framework backend, and a MySQL database. The main idea of the system is to connect stock, documents, and finance records together so the user can follow the business workflow from purchase to sale and payment.

5.2 Objective Summary

The project objectives focused on three main goals: recording purchase and sales transactions with status and document attachment support, showing useful inventory stock information, and adding an AI chatbot that can answer supported questions from backend data.

The implemented system meets these goals within the current project scope. Purchases and sales support status tracking and documents. Inventory stock is calculated from received purchases and committed sales. The assistant can answer supported read-only questions, but it cannot create or edit records.

5.3 System Design Summary

The system was designed as a three-layer web application. The frontend handles the pages and user interaction. The backend handles APIs, validation, stock rules,

eligibility rules, dashboard summaries, and assistant context. The database stores the actual business records using relational tables.

This design was useful because it keeps important business rules on the back-end. The frontend can make the system easier to use, but the backend still controls the rules that protect stock, billing, supplier payments, and credit notes.

5.4 Implementation Summary

The frontend implementation includes pages for dashboard, inventory, AI chat, AI report, purchases, payment batches, quotations, sales, billing notes, credit notes, products, categories, suppliers, customers, user access, and activity logs. It also includes a shared API client, authentication handling, permission-based navigation, pagination support, filters, modals, document references, bilingual text, and mock-data fallback for development.

The backend implementation includes Django models, serializers, viewsets, service functions, dashboard endpoints, lookup endpoints, eligibility endpoints, authentication endpoints, user and role management endpoints, activity logging, AI report generation, management commands, and backend tests. The most important backend logic includes stock calculation, stock allocation, sale stock validation, purchase payable calculation, finance eligibility, permission enforcement, assistant answer preparation, and AI report context building.

5.5 Result Summary

The final result is a working inventory management system that connects the main trading workflows. A received purchase can increase stock, a committed sale can reduce stock, a billing note can track money expected from a customer, and a payment batch can track money that needs to be paid to a supplier.

The dashboard and inventory pages make the system easier to review because they summarize stock and finance information in one place. The assistant adds another way to ask about supported operational information, but it stays within a narrow read-only scope. The AI report page extends this further by letting the user generate printable

supplier, customer, or product reports with metrics, charts, and business analysis.

5.6 Key Findings

The main finding from this project is that stock should be calculated from transactions instead of being manually typed into a product field. Received purchases and committed sales give a clearer picture of what stock is really available.

Another finding is that backend validation is necessary. The frontend can guide the user, but the backend must still reject invalid stock commitments and invalid finance document selections.

The project also showed that historical snapshots are important. Old business documents should still show the product name, SKU, supplier name, customer name, prices, and totals that existed when the document was created.

5.7 Obstacles and Challenges

The main challenge was connecting many workflows without creating inconsistent data. A change in purchase status can affect stock and supplier payment. A change in sale status can affect stock, billing, and credit notes.

Another challenge was keeping the system understandable for daily use while still applying strict backend rules. The user interface needed to stay practical, but the backend still had to protect the data.

The last challenge was preparing enough final evidence for the report. This was addressed by recording selected backend test runs, validating the reorder point and AI-related behavior, and collecting scenario test and questionnaire results from 10 anonymized testers.

5.8 Future Work

Future work should stay focused on practical improvements:

1. Continue user testing with a larger and more diverse group of SME users.
2. Review the final staff roles and permission assignments with a real business before production deployment.

3. Add clearer audit history details for important status changes such as purchase receiving, sale delivery, billing receipt, and supplier payment.
4. Improve deployment testing with larger data and more users before real production use.

REFERENCES

[1] K. Bishop. Qtier-Rapor: Managing spreadsheet systems and improving corporate performance, compliance and governance. arXiv:0803.0011, 2008. https://arxiv.org/abs/0803.0011.

[2] K. Lappalainen and S. Narayanan. Aisha: A custom AI library chatbot using the ChatGPT API. *Journal of Library and Information Services in Distance Learning*, pages 1--15, 2023. doi: 10.1080/19322909.2023.2221477.

[3] H. Mittal and M. Arora. A full-stack web solution for online FMCG (fast-moving consumer goods) management system using MERN: Design, implementation and evaluation. *International Journal of Latest Trends in Engineering, Management and Applied Sciences (IJLTEMAS)*, 2024.

[4] M. Muller. *Essentials of Inventory Management*. AMACOM, New York, NY, 2 edition, 2011.

[5] Office of Management and Budget. *North American Industry Classification System: United States, 2022*. Executive Office of the President, Office of Management and Budget, 2022. Sector 42, Wholesale Trade.

[6] The Times. Sole traders still not prepared for digital tax returns in April. Published 1 December 2025. Reports Sage research on sole traders' use of spreadsheets, bank statements, and pen-and-paper accounting.